NVIDIA 开发者日（Developer Day）中国站 · 大语言模型训练与推理分会场 · 2026-10-16

大模型训练与推理 · 全天技术讲义

Training & Inference for Large Models — A One-Sitting Handout

上午主论坛 + 下午 4 场分享 / 5 位讲者（13:31–16:35）｜依据：96 张投影片照片 OCR + 现场录音转写（含 4 段补录视频）｜配套长文档：《内容整理与总结分析》《讲稿整理（录音对照）》

一页鸟瞰

全天时间轴

时段	内容	关键产出
09:45–12:10	主论坛：开场致辞 → **四大 scaling law**（英文演讲）→ Cosmos / 世界模型 → Nemotron 3 / Megatron → 圆桌"从开放到可控"	全天基调：**算力** → **智能**的四条 scaling law；"open = 权重 + 数据 + 环境 + 配方"
13:31–14:11	**第 1 场 MegaMoE**（孙方宇 1–15 页、张毅 16–34 页）	把 MoE 四段融进**一个持久内核**：理论上界 1.42× → **1.92×**，实测几何平均 **1.16×/1.30×**
14:12–15:00	**第 2 场 大规模训练实践**（白洪潇，NVIDIA Megatron）	四步 playbook：**64K +59%、1M +32%**
15:00–15:44	**第 3 场 KV Cache 压缩**（徐文杰）	三条路线：TurboQuant **≥4.5×**、KVTC **8–64×**、nvCOMP-ANS **≈1.4×**
15:44–16:27	**第 4 场 Dynamo / 智能体推理**（卢祥龙，NVIDIA 解决方案与工程）	缓存感知路由 + 索引 **170M ops/s** + 会话网关：**E2E −51%、TTFT p50 −87%**

三句话抓住全天

1. **算子层不要划边界**：MoE 的最优解不是"更快的通信"或"更快的 GEMM"，而是让 dispatch / FC1 / FC2 / combine 在同一个内核里按专家波次交织——再看清"融合税"（SMEM 借用、TMA 干扰、精度不对称），用 30+ 项微架构优化逐条偿还。
2. **系统层压缩状态**：长上下文时代，瓶颈从"算"转向"存"与"搬"。KV 压缩（在线量化 / PCA / 无损熵编码）+ 缓存感知路由 + 会话调度，是在**不换硬件**的前提下换性能的三件套。
3. **度量层重写 SLA**：当负载从"请求"变成"会话轨迹"，TTFT/TPOT 已不够；trajectory 级 E2E、首个答案 token 时间、满足 SLA 的并发 agent 数，才是新的优化目标函数。

一句话记住每位讲者

讲者	一句话判断（其口头的核心主张）
孙方宇 / 张毅	"**融合不是免费的**"——所以要先算清代价，再逐项还债；小 token 场景的关键是把 token 挪到 N 维（swapAB）。
白洪潇	"**并行与显存策略共同决定可行性**"；优化顺序比优化本身更重要；且**训练侧的经验不能直接搬到推理**。
徐文杰	选压缩先看**精度**、再看形态、最后看你要缓解哪一种压力；"**免费的午餐（无损）也是有代价的**"。
卢祥龙	把**状态**（KV）当作有位置的资源来调度；与其纠结淘汰策略，不如"把本地缓存设小、用一个大的共享池"。

二、上午：四条 scaling law 与"开放"的定义

2.1 讲者提出的四个 scaling law（10:54–11:07 补录段）

#	scaling law	要点
1	**预训练**	用全互联网/全语言/全领域数据 + 用 AI 提纯数据；"**数据变聪明，让预训练（同等硬件）快了 4 倍**"，并把预训练数据集公开以加速生态
2	**后训练**	"巨大的加速计算问题"：智能体要在**百万级 sandbox** 与各类环境中训练，部分环境需**内置模拟器**，模型才能真正理解世界
3	**测试时计算**	回答一个问题**需要多少 token** 本身就是加速计算问题；算得快但用太多 token 就是浪费（提到受 GLM 5.2→5.3 启发）
4	**数值/稀疏（推断）**	第四条的完整表述落在素材缺口处；前后文为"20 年前几乎全是 FP32，如今大部分训练是 8 位算术，等于 4×"`(?)`

2.2 "开放"到底开放什么

- **不只是权重**：Nemotron 同时给 **weights + data + environments + software + recipes**；"open = 权重 + 数据 + 配方"这一说法在上午两场中重复出现。
- 讲者特别强调：做开放**不是因为好玩**，而是"**对我们的业务和未来是必要的**"，与他们对 CUDA 的长期承诺同一量级。
- 一个具体的架构因果：**NVLink-72（72 卡全速互通、低延迟）之所以这样设计，是因为判断 MoE 会改变 AI 的构建与部署方式**——这条判断正是下午第 1 场"MoE 通信-计算融合"的硬件前提。

2.3 上午其余三条线索（对下午有用）

1. **CUDA tile programming**：可以用 Python / C++ / Rust 写 tile 程序；"tile 正在成为一等公民"——下午第 1 场的 `256×256 tile`、`task tile`、`M64×N128` 等术语都建立在这个抽象之上。
2. **Nemotron Ultra 的训练流水**：550B 参数、20T token 预训练 → 扩展到 **1M 上下文**得 base → teacher 模型生成 SFT 数据 → 15 个可验证奖励环境做 RLVR → **多域 on-policy 蒸馏**合成最终模型。这条"数据飞轮"解释了下午为什么反复强调"measured / profile-driven"。
3. **圆桌共识**：harness（智能体执行框架）的终局可能是"**OS**"；对 2029–2030 的三个"1"（成本、能耗等）与"卡和电"的长期判断——与下午第 4 场的"每 token 成本"视角同源。

三、第 1 场 MegaMoE：让通信与计算住进同一个内核

3.1 问题与上界

- MoE 层天然四段：**dispatch（all-to-all 发出）→ FC1 + SwiGLU → FC2 → combine（收回）**；串联执行时通信与计算互不重叠。
- 三条时间轴：朴素顺序 = **1.00×**；Comet 式"通信/计算两路流水" = **1.42×**；按**专家波次**把通信切段与 `L1/L2` 计算交替填满 = **1.92×**（理论上界）。

3.2 融合的三笔"税"

税目	具体表现
借 SMEM	通信要占用计算的 shared memory，挤占主循环预算
干扰 TMA	通信流量压低主循环 TMA 效率（NVLink 利用率与 GEMM 效率互相牵制）
精度不对称	NVFP4 让 L1/L2 耗时减半，但 combine 仍是 **bf16** → NVLink 流量占比被放大；**FP8 下 combine 还能藏进 L2 epilogue，FP4 下藏不进去**

3.3 十项关键手段（按投影片编号）

#	手段	关键点
GEMM	**256×256 tile**（原 256×192）	TMEM 只有 512 列 → **overlap ACC** 技巧逼近两段式累加
GEMM	**swapAB 动态 tileN**	WGMMA 形状 `m64nNk32`：非 swapAB 下 token tile 最小 64（**8 个 token 浪费 56 行**）；swap 后 `M=channels`、`N=tokens(8…128)`，**tile 利用率最高 8×**；swap 只是 stride view、零拷贝，expert 维变 `L` mode 使 TMA descriptor tile-invariant
GEMM	**TMEM 内 epilogue 转置**	从 `16×8→32×4` 的小技巧出发，把 LDTM/置换/STTM 建模为**位级置换**，用 **Cayley/BFS 搜索 + 模拟器验证**得到完整转置（人 + LLM 协作）
同步	**batch-commit / async UREDGR**	跨卡写 flag 需 fence + barrier 两条指令；合并提交、异步归约"不阻塞后续发射"
调度	**单 dispatch warp**	实测**1 个 > 2 个 > 4 个**：约 **132 行在飞**已超过 NVLink 带宽时延积，队列更深反而拉长往返
调度	**折叠 producer warp-group**	让双 WG 协作可行：blockwise 512–32768 全部切 `M64N256`，**+11%~+30%，几何平均 +15.8%**
同步	**提前发布 `fc1_done`**	发布点上移，FC2 更早拿到就绪信号，缩短等待
调度	**调度分组 + 消除自旋**	小组(1–3 专家)时 FC2 紧追 FC1 → producer **约 25% 时间在自旋**；把组扩到 **264(≈6 专家)** 后自旋消失
epilogue	**16 字节 FC2 写出**	4×4 butterfly transpose（4 次 shuffle）让每条 lane 持 8 个连续列 → `8×STG.32` 变 **1× `STG.128`**
架构	**一个 kernel 做所有事**	4 dispatch warp 经 NVSHMEM symmetric heap 拉 token → FP8 pool 零拷贝当激活 → GMEM counter 握手 → combine 写回；**3 executor × 2 reduce 落点 = 6 种模式**；20 warps / 640 threads，无独立 MMA warpgroup（epilogue WG 自己发 WGMMA，acc 常驻 RMEM）

3.4 数据与判断

口径	Per-Tensor FP8	Blockwise FP8
小 token（≤512）	1.02–1.11×	差异很小
大 token（≥1024）	1.24–1.35×（峰值 **1.35×** @8192）	1.33–1.72×（峰值 **1.72×** @32768）
几何平均	**1.16×**	**1.30×**
口头档位值	小 token 约 **3%**、大 token **24%**	32768 档 **72%**

> **讲者的判断**：小 token 时两者接近，因为 **launch 与通信主导、两种配置都是 memory-bound**；只有 GEMM 占比上升后，单内核"省 launch、省中间张量往返、通信与计算交织"的优势才显现。**收益是形状相关的，不是普遍 1.9×。**

四、第 2 场 大规模训练：装得下，再跑得快

4.1 两个挑战与四步 playbook

Make it fit（装得下）	Make it fast（跑得快）
超大参数量、模型状态与 HBM 容量、长上下文、并行布局与拓扑、特性兼容性	kernel 与 launch 开销、重算与 offload 代价、通信与负载不均衡

步骤	做什么	模块级	端到端
1 运行开销	64K 的主要问题是"**大量 GPU idle**"：kernel fusion + **CUDA Graphs**（packed sequence 用 padding 变静态后捕获）	—	64K **>30%**、1M **<3%**（口头：1M 显存太紧开不了 CUDA Graphs）
2 混合注意力	①稀疏注意力"左对齐有效索引、只扫前 L 个 slot"；②Indexer **一个内核内打分+选择**（Q·K 量化 FP8）；③Group-O 的 **8 个小 GEMM 合成一个 batched GEMM**	稀疏 **25–30%**、FP8 Indexer **25–35%**、Group-O **50–60%**	64K **+4.6%**、1M **+9.0%**
3 MoE	MoE 中 **30–50%** 是 EP 通信；跨 batch EP 重叠（跨 NVLink 域）或 **megakernel 细粒度重叠**；融合执行 + ring buffer 复用	—	64K **+14.0%**、1M **+4.6%**
4 CP 负载均衡	QK score 随序列长度**平方**增长 → 512K/CP32 下同组 rank 工时为 **53.1 → 18.4 ms**；用 **zigzag** 把最轻段与最重段配到同一 rank	—	1M **+12.2%**；**64K 无收益**（均衡本身有开销）

整条 playbook 叠加后：**64K +59%、1M +32%。**

4.2 起点与边界

- 模型：**DeepSeek-V4-Pro，1.6T 总参数 / 49B 每 token 激活**；结构含 `CSA/HCA` 混合注意力、`Indexer`、`MTP Modules`；预训练从 `4K→16K→64K→1M`，数据从 dense 过渡到 sparse。
- 布局三原则：**模型规模定初始布局**（专家优先 EP、跨域 PP、必要时 TP）、**上下文长度定 CP**、**拓扑定放置**（EP/TP/CP 尽量收进 NVLink 域）；实例 **PP4 + EP64**；64K 用 `CP16`+细粒度重算+offload，1M 用 `CP64`+全量重算（每 rank token 4096 → 16384，4×）。
- 先证后调：用 **4K 上下文的 sparse 代理模型**验证特性，再靠 profile 决策（"模型 80% 以上时间在 attention + MoE"）。

> **讲者的判断**：① **训练 ≠ 推理**——"推理方向可能完全不一样，这些东西没法直接迁移过去"；② CUDA Graphs 有代价——"可能带来几 G 甚至十几 G 额外显存"，与卡型无关；③ FP8 Indexer 的精度损失**长时训练未验证**，DeepSeek 官方实现的精度"其实没说"，属不确定状态。

五、第 3 场 KV Cache：把状态压小，把底线守住

5.1 三重瓶颈与两条集成路径

瓶颈	因果
GPU 容量	能装下的活跃请求变少 → 有效 batch ↓、可用上下文 ↓
HBM 带宽	每生成一个 token 都要重读历史 → 每步读字节数 ↑
系统 I/O	溢出时 KV 在 GPU ↔ CPU/SSD/远端搬运 → **TPOT** ↑、prefill/decode 被拖慢

- **推理引擎内**（vLLM/SGLang/TRT-LLM）：压缩后 KV 常驻 GPU、对引擎透明；挑战是需与 attention 融合、与引擎/模型强绑定。
- **KV 缓存后端**（LMCache/FlexKV/Mooncake）：注意力前先还原；收益是存储/传输↓ + 前缀命中潜力；代价是**解压开销、无直接 TPOT 收益。**
- **判定规则**：`KV-read savings > decompression overhead` → 放推理引擎，**否则退回后端。**

5.2 三条路线与关键数字

路线	机制	压缩/收益	代价
TurboQuant（在线量化）	随机旋转 → Lloyd-Max codebook（码心 −1.2/−0.4/+0.4/+1.2）→ 逆旋转 → 可选 QJL 修正；每向量只存质心索引 + 残差符号位 + 标量范数；布局改**页内字段平面化**（16 token/页、**2,144 B/KV head**），32 B 扇区有效字节 **3.68 B → 29.75 B**（≈8×）；解码用 64-token tile + FP32 running state	**≥4.5×** 名义载荷；解码吞吐短上下文 1.25–1.28×、长上下文 **2.5–4.8×**（峰值 **4.84×**）；生产甜点约 **3.6×**	需定制解码内核；基线在长上下文多处 **FA OOM**
KVTC（存储/传输）	跨层/头拼接 → PCA → 动态量化（重要主成分给更多比特）→ 可选 DEFLATE；压缩单元一个 token；**流水化解压** + 可插拔 codec	**8×–64×**；16× CR 精度损失 **≤0.8 分**（3 模型 × 5 基准）；单请求 TTFT **−5.5% ~ −10.8%**；96 GiB 压力下吞吐 **2.49×**、TTFT **−64%**、命中率 **11.9% → 74.7%**	粒度不匹配 + dequant/逆 PCA 太重；容量充足时几乎无开销也无收益
nvCOMP-ANS（无损）	ANS 熵编码，可**精确还原**	TTFT **≈1.4×(BF16) / ≈1.1×(FP8)**；P90 最高 **+18.7%**、均值 **+8.2%**（低并发收益最大）	无损 → 压缩率有限；**解压占 SM**、并发越高越不稳（波动约 −5.6% ~ +5%）

讲者的判断：① 选型顺序是**精度 → 形态（存储型/计算型、能否融合）→ 优化目标（HBM / 容量 / 系统 I/O）**；② "**免费的午餐（无损）也是有代价的**"：解压要占 SM（解压带宽约 50 GB/s），高并发下系统稳态变复杂、TTFT 反而下降；③ 任何 codec 上线后都要**重新度量质量、TTFT、TPOT、吞吐与命中率。**

六、第 4 场 Dynamo 与智能体推理：从"算得快"到"找得准、排得开"

6.1 路由：一个成本函数 + 一张位置地图

- 请求路径：`Token → Block Hash → KV Indexer(前缀命中) → Slot Tracker(活跃 prefill/decode) → Cost Function → 最低成本 worker`，worker 反向推 `KV Events` 更新索引。
- **`Cost = Prefill 成本 + Decode 成本 +（可选）活跃请求成本`**；"缓存告诉我们能省多少，负载告诉我们还能接多少"。
- 索引键是 **`(position, block_hash)`**（只比内容会误判前缀连续）；Flash Indexer 只回答"哪里有缓存"，但要同时满足**查询快**与**状态新**。
- 六代演进：`Nested Dict 385K`（O(W×D)）→ RustActor → Inverted Index → `Radix Tree 4M`（tree = prefix context；双键 local_hash/seq_hash）→ Concurrent Radix → **`Positional Indexer + Jump Search 170M`（O(D/J+W)，checkpoint 跳段，Single 直接命中、Multi 才 lazy 算 seq_hash）**；累计 **440× / 42×**。

6.2 容量归因：为什么是"存不下"而不是"算得慢"

事实	数值
每 token KV	**55.9 KB**（MLA 43.9 + DSA 9.8 量级拆分）
1 GB / 单节点 8 卡	TP8 **2.2K** token/GB vs DEP **17.9K**；4 节点合计 **5.0M vs 30.6M**
会话并发上限	TP8 8× 复制只能约 **33 个会话**；DEP 约 **200 个**；**cc 过 32 就雪崩**（口头）
三级存储	GPU G1 ≈1710 GB、主机 G2 ≈2048 GB、G3 ≈1024 GB；**G3 只装得下约 41 分钟**写入
命中率	理想 vs 实际差 **−3.9 pt**（**95.8% → 91.9%**）；重算占 **8.2%**

6.3 会话网关 ThunderAgent

- 全局控制回路：**Δt = 5 s 一拍**，`util = 活跃 program 上下文量 ÷ GPU KV 池`；门限 **0.80 / 0.85 / 0.95**；恢复顺序按"上下文最小、重算最便宜"；**>1800 s 强制恢复**（防饿死）；暂停命中冷区则 offload（多 program 共享同一份 KV）。
- 实测（SWE-Bench Lite、GLM-5.2、8×Hopper、并发 32）：**E2E 均值 −51%、TTFT p50 −87%、TPOT 减半、吞吐 ×1.6**；RL rollout 场景约 **2.5×**、而 rollout 占训练时间约 **70%**（口头）。实测节奏："**一小时 pause 45 次、resume 77 次**"（口头）。

> **讲者的判断**：① 智能体时代的 SLA 必须换成 **trajectory 级**（trajectory E2E、首个答案 token 时间、AA-AgentPerf、全局命中率 >90%）；② **Hard affinity 会把某些 rank 长期顶到 90%+**，缓存亲和必须与负载均衡联立求解；③ 落地建议——小版本门槛（开源 ≥1.5/1.5.1）、"把本地 L2 设小甚至 0、用大的共享 CPU/SSD 池让节点互相可见，就不会有不命中"。

七、速查：数字与出处

数字	含义	出处
1.42× / 1.92×	Comet 式 vs MegaMoE 式融合的理论上界	页 1
256×256、8×	大 tile（原 256×192）；swapAB 的 tile 利用率上限	页 6、19
20 warps / 640 threads、256	warp 编制；epilogue 寄存器上限	页 18
132 行在飞、1>2>4	单 dispatch warp 的带宽时延积；warp 数实测排序	页 27
+11%~+30% / +15.8%	折叠 producer WG 后的 blockwise 收益（区间/几何平均）	页 28
≈25% → 0	`fc1_done` 自旋占比（组扩到 264 后消失）	页 31
1.16× / 1.35×；1.30× / 1.72×	token sweep 几何平均 / 峰值（per-tensor；blockwise）	页 33
3% / 24% / 72%	口头档位加速比（小 token / 大 token / 32768）	第 1 场口头
1.6T / 49B	DeepSeek-V4-Pro 总参数 / 激活参数	页 37
PP4+EP64；CP16→CP64；4096→16384	初始并行布局；上下文切换；每 rank token	页 39–42
+59% / +32%	整条 playbook 的吞吐提升（64K / 1M）	页 36
25–30% / 25–35% / 50–60%	稀疏注意力 / FP8 Indexer / Group-O 的模块级加速	页 47
+4.6% / +9.0%；+14.0% / +4.6%；+12.2%	STEP2 / STEP3 / STEP4 的端到端收益（64K / 1M）	页 47、49、51
30%–50%	MoE 中 EP 通信的占比	页 48
4.5× / 3.6×；29.75 B；4.84×	TurboQuant 极限/甜点；扇区有效字节；解码峰值	页 57–60
8×–64×；≤0.8 pt；−5.5%~−10.8%；2.49×	KVTC 压缩率、精度、TTFT、压力下吞吐	页 61–64
≈1.4×/1.1×；+18.7%/+8.2%	nvCOMP-ANS 的 TTFT 收益（峰值/均值）	页 65–66
385K→4M→170M；440×/42×	Flash Indexer 六代吞吐与累计提升	页 73–76
55.9 KB；33 vs 200；1.26M/7.6M	每 token KV；TP8 vs DEP 会话数；token 容量	页 85
95.8%→91.9%；8.2%	实际命中率缺口；重算占比	页 86
−51% / −87% / ×1.6	ThunderAgent 的 E2E / TTFT p50 / 吞吐	页 93
393 会话 / 98,827 请求 / ISL 219K / p99≈900K	AgentX trace 规模	页 80–81

> 标注"口头"的数字只出现在录音转写中，投影片上没有；其余为投影片印刷值。两处已知的口径差异见《讲稿整理》4.7 节。

八、术语速查与待验证问题

8.1 术语（20 条）

术语	一句话
MegaMoE	把 MoE 的 dispatch/FC1/FC2/combine 融进一个持久内核的实现
swapAB	交换 A/B 操作数，把 token 映射到 N 维，使小 token 也能填满 tile
WGMMA / TMEM / TMA	warp-group 级 MMA；Blackwell 张量存储；异步张量搬运
R2S	register → shared memory（epilogue 内的搬运）
NVFP4 / MXFP8	4 位 / 8 位缩放（block）浮点格式；combine 仍常需 bf16
EP / CP / PP / TP / DP	专家 / 上下文 / 流水 / 张量 / 数据并行
NVSHMEM	GPU 发起的一侧通信库（symmetric heap、dispatch/combine）
DeepGEMM / CUTLASS / CuTeDSL	量化 GEMM 库 / 内核模板库 / CuTe 的 Python DSL
TurboQuant	在线 KV 量化（旋转 + Lloyd-Max codebook + QJL 修正）
KVTC	面向存储/传输的 KV 压缩（跨层 PCA + 动态量化）
nvCOMP-ANS	无损熵编码的 KV 压缩/传输
Dynamo	NVIDIA 推理服务框架（KV Router、Indexer、provider plugin）
KV Router / Flash Indexer	缓存感知路由 / 前缀索引服务（位置→worker 地图）
DEP	与 KV offload/亲和路由一起出现的并行/部署形态
TTFT / TPOT / ISL	首 token 时间 / 每输出 token 时间 / 输入长度
AA-AgentPerf	满足 SLA 的最大并发 agent 数
ThunderAgent	会话网关（program 状态机 + 5 s 全局控制回路）
MoK	第 2 场引用的 Cursor 侧 MoE 工作（"Mixture-of-Kittens"，`(?)`）
FA OOM / capacity-skipped	基线 FlashAttention 因容量不足被跳过，不计入对比
trajectory SLA	以整条智能体会话轨迹为单位的服务等级目标

8.2 值得继续追问的 8 个问题

1. MegaMoE 在 Blackwell（SM100）与 Hopper（SM90）上的收益差多少？`swapAB` 在 SM100 上是否同样成立？
2. 1.92× 上界里"通信与计算可完全重叠"的假设有多强？实际重叠率是多少？
3. per_tensor 与 blockwise 的选择准则是什么（专家数、token 分布、combine 精度）？
4. "EP 通信占 MoE 30–50%" 随规模如何变化？到万卡是否仍成立？
5. TurboQuant 的精度损失在大 batch 长上下文下如何累积？与 KVTC 的 PCA 路线能否叠加？
6. KV Router 的两类信号权重怎么标定？负载突增时如何避免"缓存热节点雪崩"？
7. ThunderAgent 的 5 s 控制周期与真实会话间隔（60 s–46.7 h）如何匹配？更长的暂停是否应直接 offload？
8. 轨迹级 SLA 在多租户下如何定价与配额？"满足 SLA 的并发 agent 数"会不会成为新的容量单位？

8.3 使用说明

- 本讲义是**浓缩版**，每条结论的原文、图表与不确定标记见两份长文档：`大模型训练与推理_内容整理与总结分析.pdf`（投影片整理）与 `大模型训练与推理_讲稿整理（录音对照）.pdf`（录音对照、逐页口头要点、四场问答）。
- 数字口径：投影片印刷值以页号标注；只在录音中出现的以"口头"标注；OCR 存疑处标 `(?)`。
- 会议身份、讲者姓名由现场录音确认：NVIDIA 开发者日（Developer Day）中国站，"大语言模型训练与推理"分会场，4 场分享 / 5 位讲者（孙方宇、张毅 / 白洪潇 / 徐文杰 / 卢祥龙）。